Underwater Robot Trajectory Viewer

Mathematical Foundations and Implementation Notes

Geovani

September 30, 2026

Contents

1 Overview **1**

2 Input data **2**

3 Coordinate frames **2**
3.1 Geodetic frame . . . 2
3.2 ECEF frame . . . 2
3.3 Local NED frame . . . 2
3.4 Depth . . . 3

4 WGS84 ellipsoid **3**
4.1 Prime-vertical radius of curvature . . . 3
4.2 Meridian radius of curvature . . . 3

5 Geodetic → ECEF **3**

6 ECEF → local NED **4**
6.1 Basis vectors . . . 4
6.2 Rotation matrix . . . 4
6.3 Transformation . . . 4
6.4 First-order check . . . 4
6.5 Why D is not exactly $h_{\text{ref}} - h$ . . . 4

7 Choice of common origin **4**

8 Implementation **5**

9 Verification **5**

10 Limitations and next steps **5**

1 Overview

The project displays the trajectories of three underwater robots (`robot1`, `robot4`, `robot7`) in a single three-dimensional Cartesian frame with depth. Two parallel implementations share the same mathematics:

- **Python / Plotly** (`python/viewer.py`): computes the conversion with NumPy and writes a self-contained HTML file.

- **C++ / HTML** (`cpp/`, `web/`): object-oriented C++17 classes perform the conversion and export JSON; an HTML page (Plotly.js) fetches that JSON and only renders.

Both produce the same coordinates (maximum difference 3.9 μm, Section 9).

2 Input data

Each CSV row is a ROS `nav_msgs/Odometry` sample. The columns used are `Time`, `pose.pose.position.x`, `.y`, `.z`.

Interpretation of the position columns. Although named x, y, z, the values are *geodetic* coordinates:

$$x \equiv \varphi \text{ [deg]} \approx 41.45, \qquad y \equiv \lambda \text{ [deg]} \approx -8.81, \qquad z \equiv h \text{ [m]}, \tag{1}$$

with latitude φ, longitude λ and height h above the WGS84 ellipsoid. Over each mission h starts near 56 m (robot at the surface) and decreases to about 25–30 m (the dive), which supports reading z as a height that decreases with depth.

Assumptions to be confirmed.

1. The `frame_id` fields (`world_ned` for robots 1 and 4, `WGS84` for robot 7) are inconsistent with the values, which are geodetic for all three robots; all three files are therefore treated identically.
2. The water surface is at height h_{ref}, taken as the mean over robots of the median of the first 50 samples ($h_{\text{ref}} = 56.460$ m). It can be overridden (`--h-ref` in Python).

Robot	Samples	Dur. [min]	Δt [s]	North [m]	East [m]	Max depth [m]
Robot 1	9537	84.8	0.534	[13.1, 152.2]	[1.0, 146.5]	30.3
Robot 4	5615	49.9	0.534	[23.8, 182.6]	[14.3, 188.5]	28.3
Robot 7	15000	133.5	0.534	[-144.2, 36.9]	[-136.1, -59.1]	31.4

Table 1: Dataset summary. North/East ranges are relative to the common origin of Section 7.

3 Coordinate frames

3.1 Geodetic frame

A point is (φ, λ, h): φ is the angle between the ellipsoid normal and the equatorial plane, λ the angle from the Greenwich meridian, and h the signed distance along the normal.

3.2 ECEF frame

Earth-Centred Earth-Fixed: origin at the Earth's centre, $\boldsymbol{e}_z$ towards the north pole, $\boldsymbol{e}_x$ through $(\varphi, \lambda) = (0, 0)$, $\boldsymbol{e}_y$ completing a right-handed triad. Coordinates $\boldsymbol{p}^e = (X, Y, Z)^\top$ in metres.

3.3 Local NED frame

Given an origin $(\varphi_0, \lambda_0, h_0)$ the local tangent frame has axes North, East, Down, with Down pointing opposite to the ellipsoid normal. Coordinates $\boldsymbol{p}^n = (N, E, D)^\top$.

3.4 Depth

With the origin height set to $h_0 = h_{\text{ref}}$, the Down coordinate is (to first order) $D \approx h_{\text{ref}} - h$, i.e. the depth below the water surface. The plot shows $x = E$, $y = N$, $z = D$ with the z axis reversed so that deeper points appear lower.

4 WGS84 ellipsoid

$$a = 6\,378\,137 \text{ m}, \qquad f = \frac{1}{298.257223563}, \qquad b = a(1-f), \qquad e^2 = f(2-f) = 1 - \frac{b^2}{a^2}. \tag{2}$$

4.1 Prime-vertical radius of curvature

Work in a meridian plane with $\rho = \sqrt{X^2 + Y^2}$ and Z. The ellipsoid satisfies $\rho^2/a^2 + Z^2/b^2 = 1$. The outward normal is proportional to the gradient $(\rho/a^2,\ Z/b^2)$, and by definition of geodetic latitude it is proportional to $(\cos\varphi, \sin\varphi)$. Hence

$$\frac{Z}{\rho} = \frac{b^2}{a^2}\tan\varphi = (1-e^2)\tan\varphi. \tag{3}$$

Substituting (3) into the ellipse equation and using $b^2 = a^2(1-e^2)$:

$$\frac{\rho^2}{a^2}\big(1 + (1-e^2)\tan^2\varphi\big) = 1 \implies \rho^2 = \frac{a^2\cos^2\varphi}{\cos^2\varphi + (1-e^2)\sin^2\varphi} = \frac{a^2\cos^2\varphi}{1 - e^2\sin^2\varphi}. \tag{4}$$

Defining

$$\boxed{N(\varphi) = \frac{a}{\sqrt{1 - e^2\sin^2\varphi}}} \tag{5}$$

gives $\rho = N\cos\varphi$ and, from (3), $Z = N(1-e^2)\sin\varphi$ on the ellipsoid surface.

4.2 Meridian radius of curvature

$$M(\varphi) = \frac{a(1-e^2)}{\big(1 - e^2\sin^2\varphi\big)^{3/2}}. \tag{6}$$

At the origin $\varphi_0 = 41.4523095°$: $N = 6\,387\,513.5$ m and $M = 6\,363\,421.7$ m.

5 Geodetic → ECEF

A point at height h is the surface point displaced by h along the unit normal $\boldsymbol{u} = (\cos\varphi\cos\lambda,\ \cos\varphi\sin\lambda,\ \sin\varphi)^\top$:

$$\boxed{\boldsymbol{p}^e = \begin{pmatrix} (N+h)\cos\varphi\cos\lambda \\ (N+h)\cos\varphi\sin\lambda \\ \big(N(1-e^2)+h\big)\sin\varphi \end{pmatrix}} \tag{7}$$

This is exact (closed form, no iteration). The inverse (ECEF → geodetic) is not needed here because the input is already geodetic.

6 ECEF → local NED

6.1 Basis vectors

At the origin (φ_0, λ_0), with $s_\varphi = \sin\varphi_0$, $c_\varphi = \cos\varphi_0$, $s_\lambda = \sin\lambda_0$, $c_\lambda = \cos\lambda_0$:

$$\boldsymbol{u} = (c_\varphi c_\lambda,\ c_\varphi s_\lambda,\ s_\varphi)^\top \qquad \text{(Up: ellipsoid normal)} \tag{8}$$

$$\hat{\boldsymbol{e}} = (-s_\lambda,\ c_\lambda,\ 0)^\top \qquad \text{(East: } \partial \boldsymbol{p}^{\,e}/\partial\lambda \text{ normalised)} \tag{9}$$

$$\hat{\boldsymbol{n}} = (-s_\varphi c_\lambda,\ -s_\varphi s_\lambda,\ c_\varphi)^\top \qquad \text{(North: } \partial \boldsymbol{p}^{\,e}/\partial\varphi \text{ normalised)} \tag{10}$$

$$\hat{\boldsymbol{d}} = -\boldsymbol{u} = (-c_\varphi c_\lambda,\ -c_\varphi s_\lambda,\ -s_\varphi)^\top \qquad \text{(Down)} \tag{11}$$

These follow from differentiating (7):

$$\frac{\partial \boldsymbol{p}^{\,e}}{\partial\varphi} = (M+h)\,\hat{\boldsymbol{n}}, \qquad \frac{\partial \boldsymbol{p}^{\,e}}{\partial\lambda} = (N+h)\cos\varphi\,\hat{\boldsymbol{e}}, \qquad \frac{\partial \boldsymbol{p}^{\,e}}{\partial h} = \boldsymbol{u}. \tag{12}$$

The triad is right-handed: $\hat{\boldsymbol{n}} \times \hat{\boldsymbol{e}} = \hat{\boldsymbol{d}}$. Verification of the third component: $(\hat{\boldsymbol{n}} \times \hat{\boldsymbol{e}})_z = n_x e_y - n_y e_x = -s_\varphi c_\lambda^2 - s_\varphi s_\lambda^2 = -s_\varphi$.

6.2 Rotation matrix

Stacking the basis vectors as rows gives the rotation from ECEF to NED,

$$\boxed{\boldsymbol{R}_e^n = \begin{pmatrix} \hat{\boldsymbol{n}}^\top \\ \hat{\boldsymbol{e}}^\top \\ \hat{\boldsymbol{d}}^\top \end{pmatrix} = \begin{pmatrix} -s_\varphi c_\lambda & -s_\varphi s_\lambda & c_\varphi \\ -s_\lambda & c_\lambda & 0 \\ -c_\varphi c_\lambda & -c_\varphi s_\lambda & -s_\varphi \end{pmatrix}} \tag{13}$$

Because its rows are orthonormal, $\boldsymbol{R}_e^n (\boldsymbol{R}_e^n)^\top = \boldsymbol{I}$ and $\det \boldsymbol{R}_e^n = +1$; the inverse is the transpose.

6.3 Transformation

$$\boxed{\boldsymbol{p}^{\,n} = \boldsymbol{R}_e^n \big(\boldsymbol{p}^{\,e} - \boldsymbol{p}_0^{\,e}\big)}, \qquad \boldsymbol{p}_0^{\,e} = \boldsymbol{p}^{\,e}(\varphi_0, \lambda_0, h_0), \qquad \text{inverse: } \boldsymbol{p}^{\,e} = \boldsymbol{p}_0^{\,e} + (\boldsymbol{R}_e^n)^\top \boldsymbol{p}^{\,n}. \tag{14}$$

6.4 First-order check

For small offsets, (12) gives the local linearisation

$$N \approx (M+h_0)\,\Delta\varphi, \qquad E \approx (N_0+h_0)\cos\varphi_0\,\Delta\lambda, \qquad D \approx -\Delta h, \tag{15}$$

with $\Delta\varphi, \Delta\lambda$ in radians. At the origin: 1° of latitude = 111 063.6 m and 1° of longitude = 83 558.1 m. Over the three datasets the largest difference between the exact transformation and this linearisation is 4.62 mm.

6.5 Why D is not exactly $h_{\text{ref}} - h$

The tangent plane at the origin departs from the ellipsoid by roughly $s^2/(2R)$ at horizontal range s. At $s = 300$ m and $R \approx 6371$ km this is about 7.1 mm, so D and $h_{\text{ref}} - h$ differ by millimetres over the survey area; D is the geometrically consistent quantity and is the one plotted.

7 Choice of common origin

All robots must share one frame. The origin is

$$\varphi_0 = \frac{1}{K}\sum_k \varphi_k, \qquad \lambda_0 = \frac{1}{K}\sum_k \lambda_k, \qquad h_0 = h_{\text{ref}}, \tag{16}$$

averaged over all K samples of all robots, giving $(\varphi_0, \lambda_0, h_0) = (41.4523095°,\ -8.8125439°,\ 56.460\,\text{m})$. The origin only translates the plot; it does not affect relative geometry.

8 Implementation

Class (C++)	Responsibility
`geo::Vec3`, `geo::Mat3`	3-vector and 3×3 matrix (dot product, norm, matrix–vector product, transpose).
`geo::Ellipsoid`	WGS84 constants a, f, e^2 and $N(\varphi)$, eq. (5).
`geo::Geodetic`, `geo::Ned`	Plain data types for (φ, λ, h) and (N, E, D).
`geo::toEcef`	Eq. (7).
`geo::LocalFrameNed`	Builds $\boldsymbol{R}_e^n$ and $\boldsymbol{p}_0^e$ once; `toNed()` applies $\boldsymbol{p}^n = \boldsymbol{R}(\boldsymbol{p}^e - \boldsymbol{p}_0^e)$.
`Trajectory`	Named list of time-stamped geodetic samples.
`CsvTrajectoryLoader`	CSV parsing that honours quoted fields (the covariance columns contain commas) and locates columns by header name.

Table 2: C++ architecture. The reusable maths lives in the header-only `geodesy.hpp`.

Data flow.

```
CSV -> CsvTrajectoryLoader -> LocalFrameNed::toNed -> trajectories.json
    -> fetch() in index.html -> Plotly.js
```

A browser cannot execute native C++ directly; the JSON file is the bridge. A live variant (embedded HTTP server or WebAssembly) can replace it without changing the maths.

Build and run (single-line commands).

```
g++ -std=c++17 -O2 -Wall -Wextra -o traj_export cpp/main.cpp
traj_export data web/trajectories.json
cd web && python -m http.server 8000
python python/viewer.py --data data --out trajectories.html
```

Then open `http://localhost:8000` for the C++-fed page, or `trajectories.html` for the Python version.

9 Verification

The Python and C++ implementations were compared on all 30 152 samples: the maximum absolute difference in N, E or D is 3.9 μm, consistent with the six decimals written to the JSON file. Additional sanity checks: orientation quaternions have unit norm (to 10^{-15}), time stamps are strictly increasing, the median sample period is 0.534 s in all files, and no value is missing.

10 Limitations and next steps

- The meaning of z (ellipsoidal height) and the surface height h_{ref} are inferred from the data and should be confirmed.
- Orientation (quaternions) and velocities are loaded by neither implementation yet; they are needed for body-frame views.
- Time stamps of the three robots only partly overlap; a synchronised replay needs a common time axis and interpolation.